

Section 1 — DevOps: Why? (8 pts)

1.1 In **2–3 sentences**, describe the problem DevOps was invented to solve. Do not use the words "culture" or "synergy". (3 pts)

The DevOps profession was invented to bridge the divide between software developers who needed to release features quickly and operations teams who prioritized keeping servers stable. By replacing manual code handoffs with automated pipelines and shared responsibilities, DevOps allows companies to ship software rapidly without breaking production systems.

1.2 A company deploys once every 4 months. Each release takes a weekend and roughly 1 in 3 releases gets rolled back. List **three concrete problems** this causes for (a) the business and (b) the engineers. (3 pts)

For the Business	For the Engineers
Slower Time-to-Market: Features reach users 4 months late, losing market edge to competitors.	Burnout: Weekend deployments destroy work-life balance and cause high stress.
Revenue Loss: A 1/3 rollback rate causes downtime and delays expected financial returns.	Painful Debugging: Locating a bug within 4 months of accumulated code changes is extremely difficult.
High Operational Risk: Deploying massive changes at once creates unpredictable failures that damage customer trust.	Merge Conflicts: Integrating long-lived branches leads to code collisions and context-switching fatigue.

1.3 True or false, and explain in one line each: (2 pts)

#	Statement	T/F	Why
a	DevOps means "developers do operations work too."	✗	DevOps is not about forcing developers to do operations work. It is about cross-functional collaboration and automation between Dev and Ops teams.
b	If you have a CI/CD pipeline, you are doing DevOps.	✗	Having a CI/CD pipeline means you have implemented a continuous automation tool, but DevOps is much broader than just tooling.
c	DevOps aims to make deployments boring.	✓	DevOps uses automation to make releasing code so simple and reliable that it feels like an everyday, non-eventful task.
d	DevOps is a job title.	✗	DevOps is a cultural practice, mindset, and collaborative approach between development and operations teams, not a job title.

Section 2 — The DevOps Profession (8 pts)

2.1 Fill in the table. One short phrase per cell. (4 pts)

Role	Main goal	One daily task	One tool
DevOps Engineer	Automate software deployment and speed up the release cycle between Dev and Ops.	Write or maintain CI/CD deployment pipelines	GitHub Actions
SRE	Maximize system uptime, reliability, and performance under heavy load	Write scripts to automate incident resolution	Grafana
Sysadmin	Maintain servers, OS stability, and network infrastructure	Check system logs	Linux/Bash/SSH
Platform Engineer	Build self-service infrastructure and internal developer portals (IDPs)	Develop internal CLI tools or Kubernetes templates so developers can deploy independently	Kubernetes

2.2 List **five skills** you personally need to work on to become hireable as a junior DevOps engineer. To be specific — "Linux" is not an answer, "writing a systemd unit file from scratch" is. (3 pts)

Building a GitHub Actions CI/CD pipeline
Writing a multi-stage Dockerfile
Writing basic Terraform scripts:
Writing Ansible playbooks for server configuration
Troubleshooting broken container deployments in Kubernetes

2.3 Name one thing a DevOps engineer is **not** responsible for, and say who owns it instead. (1 pt)

A DevOps engineer is not responsible for setting up physical server hardware, racking equipment, and running office network cables. It owns Sysadmins, IT Support or Data Center Technicians.

Section 3 — SRE: SLA / SLO / SLI and Incidents (14 pts)

3.1 Define **SLI**, **SLO**, and **SLA** in one sentence each, then order them from "internal target" to "legal/contractual promise". (3 pts)

SLI	A real-time metric that measures the actual, current performance or reliability of a service.
SLO	An internal target set by technical teams defining the acceptable reliability threshold a service must maintain over a period of time.
SLA	A legal or contractual agreement with customers that defines expected service performance and outlines consequences or remedies if that standard is missed

3.2 Your API has an SLA of **99.9% monthly uptime**. Calculate the allowed downtime per month (assume a 30-day month). Show the arithmetic. (2 pts)

$30(\text{day}) \times 24(\text{h/day}) \times 60(\text{min/h}) = 43200 \text{ min}$
Downtime percentage = 0.1%
 $43200 \times 0,1 / 100\% = 43.2 \text{ min}$

3.3 Complete the table (30-day month): (2 pts)

Uptime	Allowed downtime / month
99%	432 min
99.9%	43.2 min
99.99%	4.32 min

3.4 Explain the **error budget** in your own words, and answer: your monthly error budget is already 90% consumed on day 12. A developer wants to ship a risky refactor today. What do you say and why? (3 pts)

- An error budget is the amount of problems or downtime a service can have while still meeting its reliability target.
- Go ahead and deploy. Since we are on Day 12 with 90% of our error budget still intact, we have a healthy safety buffer to absorb potential issues. Error budgets exist precisely to enable smart risk-taking and innovation when system stability is high.

3.5 — Incident scenario. At 03:12 the checkout page starts returning 500s. You are on call. Write your **first 15 minutes** as a numbered timeline (aim for 5–7 steps). Your answer must include: who you notify, how you decide between *fix forward* and *rollback*, and where you write things down. (3 pts)

1.Acknowledge the alert and open a notepad	Press "Acknowledge" on PagerDuty so the team knows I am awake and working on it. Open a dedicated Slack channel (#incident-checkout) and a blank doc (Notion or Google Docs) to write down every step I take with timestamps.
2.Check logs and recent code pushes	Open Grafana and application logs to see what error message the server is giving. Check if anyone recently deployed new code right before 03:12.
3.Notify the team	Send a quick message to the engineering Slack channel: "Checkout page is giving 500 errors, currently investigating." Call the secondary backup engineer on duty if extra help is needed.
4.Decide: Rollback or Fix Forward?	If a recent deployment broke it, undo the update immediately back to the last working version or fix the code directly only if it is a tiny change that takes under 2 minutes to apply.
5.Apply the fix and test	Hit "Rollback" in GitHub Actions to bring back the old working code. Go to the checkout page and test buying an item to make sure 500 errors disappear and status returns to normal.
6.Send update and save notes for later	"Checkout is fixed and fully working again." Save all notes and timestamps in the doc so the team can write a postmortem report in the morning to prevent this from happening again.

3.6 What is a **blameless postmortem**, and name **two** things that must never appear in one. (1 pt)

A blameless postmortem is an incident review practice focused on understanding systemic failures rather than assigning individual fault. Two things that must never appear in one: 1. Individual names or personal blame (pointing out who pressed the wrong button). 2. "Human error" listed as the root cause (human error is a symptom, not the root cause).
--

Section 4 — CSR vs SSR (10 pts)

4.1 Fill in the table with  /  / "depends" plus a few words of justification. (4 pts)

	CSR	SSR
First paint speed on slow phone		
SEO out of the box		
Server CPU cost		
Works with a static host (no server)		

4.2 For each product, choose CSR or SSR and justify in one sentence: (3 pts)

a) A public news site that lives on Google traffic

SSR - server-rendered HTML provides full SEO indexability out of the box so Google crawlers can instantly index new articles.

b) An internal admin dashboard behind a login

CSR - The content is hidden behind authentication, SEO is irrelevant, making CSR the ideal choice for fast in-app navigation and low server rendering costs.

c) A marketing landing page with a contact form

SSR - Marketing pages need high search visibility and fast initial page load times.

4.3 Where does **SSG (static site generation)** fit between CSR and SSR? One short paragraph. (2 pts)

SSG (Static Site Generation) fits right between SSR and CSR by combining the SEO and initial load benefits of SSR with the zero-server overhead of CSR. Instead of generating HTML per request (SSR) or in the browser (CSR), SSG pre-renders full HTML pages at build time, allowing them to be served directly from a static host/CDN with maximum speed and security.

4.4 Open any website, view page source (), and say whether it looks like CSR or SSR — and what evidence in the HTML told you. Paste the URL. (1 pt)

<https://picsart.com/> - **SSR**

Evidence in HTML:

1. **Pre-rendered Metadata:** The HTML directly contains full title and description tags (`<title>Picsart: The AI Creative Platform</title>`) instead of an empty shell.
2. **Explicit SSR Code:** The source code includes Next.js server-side chunks (`landings-ssr/_next/...`) and explicit server-side style tags (`<style data-jss-ssr="...">`).
3. **No Empty Root Tag:** The response arrives with full HTML structure and content pre-rendered by the server, rather than a single blank tag like `<div id="root"></div>`.

Section 5 — Cloudflare & Cloudflare Pages (10 pts)

5.1 Explain what a **CDN** does, using a concrete example of a user in Yerevan requesting an image hosted in Virginia. (2 pts)

A **CDN** (Content Delivery Network) is a geographically distributed network of servers designed to cache and serve static assets (like images, CSS, or JS) to users from the server physically closest to them, minimizing latency and reducing load on the origin server.

Without a CDN: A user in Yerevan requests image.jpg. The request travels across undersea cables to the primary Origin Server in Virginia (USA) and back. Due to distance and network hops, round-trip time (RTT) is high causing the image to load slowly.

With a CDN: The image is cached on a CDN Edge Server (Point of Presence / PoP) located near Yerevan (e.g., in Yerevan or Tbilisi). When the user requests image.jpg, the CDN intercepts the request and serves the cached image directly from the nearby Edge Server. Round-trip time drops and the image loads almost instantly without reaching the Virginia server at all.

5.2 Match the Cloudflare feature to what it does: (3 pts)

Feature	What it does
DNS	Translates human-readable domain names to IP addresses
Proxy (orange cloud)	Routes user traffic through Cloudflare's global edge network before reaching your origin server, masking true server IP and providing automatic SSL, DDoS protection, and performance optimization.
WAF	Inspects incoming HTTP/HTTPS traffic at the edge to identify, block, and mitigate malicious requests
Cloudflare Pages	A hosting service that deploys and serves static websites (HTML, CSS, JavaScript, etc.) directly to users from servers around the world.
Cloudflare Workers	Lets you run small pieces of serverless code (JavaScript) close to the end-user without needing to call your main backend server.
Cache	Saves copies of your website's images and files on servers near the user so the website loads instantly.

5.3 What is the difference between **orange cloud (proxied)** and **grey cloud (DNS only)**? Give one situation where you'd deliberately choose grey. (2 pts)

The difference is that **Orange Cloud (Proxied)** routes web traffic through Cloudflare's reverse proxy to hide your origin IP, cache assets, and enforce WAF/DDoS security, whereas **Grey Cloud (DNS Only)** simply resolves the domain to your server's real IP with no proxy features applied. We must choose Grey Cloud for non-HTTP services like SSH (port 22) because Cloudflare's HTTP proxy drops direct TCP/SSH connections, which would otherwise lock us out of terminal access to our origin server.

5.4 Cloudflare Pages vs a \$5/month VPS running Nginx for a static site: give **two advantages of each**. (2 pts)

Advantages of Cloudflare Pages	Advantages of a \$5/month VPS running Nginx
Static files (images, CSS, JS) are automatically copied and served from Cloudflare's worldwide servers closest to the user. This guarantees ultra-fast page loads with zero complex setup and keeps your site online even during massive traffic surges.	Full Control & Customization: Direct root access (sudo) lets you run custom Nginx modules, non-HTTP services, custom background scripts, or local databases alongside your static files.
Zero Maintenance & Cost Efficiency: free with unlimited bandwidth while completely skipping server upkeep - no OS security patches, server manual updates, or Nginx configuration management to worry about.	Predictable Data Sovereignty: You have total control over server geographical placement, logs, and storage directories.

5.5 What is a Cloudflare Pages **preview deployment**, and why is it useful in a team? (1 pt)

What it is: A temporary, fully functional unique URL generated automatically by Cloudflare Pages whenever a developer pushes code to a Git branch or opens a Pull Request (PR).
Why it is useful in a team: It enables team members (to test and review code changes in a staging-like live environment before merging into the main production branch, catching bugs early without breaking the live website.

Section 6 — LAB A: Deploy a website to Cloudflare Pages (14 pts)

Steps

1. Create a Git repo (GitHub/GitLab) named `cf-pages-lab`.
2. Add an `index.html` that displays **your name**, **today's date**, and the text `DevOps HW1`. Add basic CSS. Optionally add a `style.css` — no framework needed.
3. Push to `main`.
4. Create a free Cloudflare account → **Workers & Pages** → **Create** → **Pages** → **Connect to Git**. 5. Select the repo, keep the default build settings (no build command for plain HTML, output directory `/`), and deploy.
6. Wait for the build to go green and open the `*.pages.dev` URL.

7. Change something visible in `index.html` , commit, push. Watch the new deployment appear. 8. Create a branch `feature/color` , change the background colour, push, and open the **preview** URL.

Deliverables

[link] *.pages.dev production URL (2 pts)

[link] Repo URL (1 pt)

[

Screenshot of the Vercel Deployments page for the project 'sargsyanush03/cf-pages-lab'. The page shows a list of deployments. The 'Production' environment has three deployments: 1. Source: 'main 243f93c changes in index.html', Deployment: 'https://5bc8e73c.cf-pages-lab-12l.pages.dev...', Status: '40 minutes ago'. 2. Source: 'feature/color 49e787d css changes are made', Deployment: 'https://d6f57ab2.cf-pages-lab-12l.pages.dev...', Status: '31 minutes ago'. 3. Source: 'main 84f7645 Update index.html', Deployment: 'https://1fb344f5.cf-pages-lab-12l.pages.dev', Status: 'an hour ago'. The 'Preview' environment has one deployment: Source: 'feature/color 49e787d css changes are made', Deployment: 'https://d6f57ab2.cf-pages-lab-12l.pages.dev...', Status: '31 minutes ago'.

] Screenshot of the Deployments list showing **at least 3** deployments (3 pts)

[link] The **preview** URL from the `feature/color` branch (3 pts)

```
[2026-09-01T17:56:51.201119Z ✨ Success! Uploaded 2 files (1.46 sec)
2026-09-01T17:56:51.201289Z
2026-09-01T17:56:51.660022Z ✨ Upload complete!
2026-09-01T17:56:53.273579Z Success: Assets published!
2026-09-01T17:56:54.458648Z Success: Your site was deployed! ] Paste the build log lines that
show the deploy succeeded (2 pts)
```

Questions

6.1 What triggered deployment #2? Which exact commit?(1 pt)

Deployment #2 was automatically triggered by a GitHub webhook event when a new commit was pushed to the main branch.

Exact Commit: 243f93c ("changes in index.html")

```
2026-09-01T18:05:42.234662Z HEAD is now at 243f93c changes in index.html
```

6.2 Production and preview URLs both work at the same time. Why is that useful, and what is the risk if a preview URL leaks? (2 pts)

It allows developers to test and review new features or UI changes on a live preview site without affecting the production environment or interrupting real users.

Risk if a preview URL leaks: Unfinished code, unannounced features, debug settings, or staging endpoints could be exposed to the public, posing security and confidentiality risks.

Section 7 — LAB B: Buy a domain and bind it to your Pages site (16 pts)

Part 1 — Purchase (4 pts)

1. Compare the **same domain name** on **three** registrars (e.g. Namecheap, Porkbun, Cloudflare Registrar, GoDaddy).
2. Record for each: **first-year price**, **renewal price**, whether **WHOIS privacy** is free, and whether **transfer-out** is free.
3. Buy the one you chose.

Registrar	1st year	Renewal	WHOIS privacy	Transfer-out
Namecheap	\$0.98	\$31.98	free	free
Cloudflare Registrar	\$4.99	\$27.70	free	free
GoDaddy	\$0.99	\$54.99	free	free

Chosen registrar + one-sentence reason:

I chose Namecheap because it offers affordable first-year domain registration with free WHOIS privacy.

7.1 Why is the **renewal** price more important than the first-year price? (1 pt)

The renewal price is more important because the first-year price is often a temporary promotion, while the renewal price is what you will pay every year after the first year.

7.2 What is **ICANN WHOIS verification**, and what happens if you ignore the confirmation email? (1 pt)

ICANN WHOIS verification is an email confirmation used to verify the contact information associated with your domain. If you ignore the confirmation email, your domain may eventually be suspended until you verify your information.

Part 2 — Bind it to Cloudflare Pages (8 pts)

1. Add the domain to Cloudflare (**Add a site**), pick the Free plan.
2. Copy the **two nameservers** Cloudflare gives you, and set them at your registrar. 3. Wait for the zone to become **Active** (minutes to hours).
4. In your Pages project → **Custom domains** → **Set up a domain** → add `yourdomain.xyz` and

www.yourdomain.xyz .

5. Confirm HTTPS works and there is no certificate warning.

Deliverables

[

PremiumDNS ?

Enable PremiumDNS protection in order to switch your domain to our PremiumDNS platform. With our PremiumDNS platform, you get 100% DNS uptime and DDoS protection at the DNS level.

NAMESERVERS



Custom DNS ▼

gerald.ns.cloudflare.com

heather.ns.cloudflare.com

 ADD NAMESERVER

REDIRECT DOMAIN



You can create redirects via your DNS provider or your Namecheap account. To perform this function from your account, you must first change your nameservers to Namecheap

]Screenshot: registrar page showing Cloudflare's nameservers set (2 pts)

[

sargsyanush.site

Monitor and configure how Cloudflare processes your web traffic with the services in the menu.

[Review Cloudflare fundamentals](#) 



Your domain is now protected by Cloudflare

Your web traffic is proxying through Cloudflare, meaning:

- Your website or app loads faster with [caching](#), [speed](#), and [traffic](#) services
- Attacks and bots are mitigated by [security](#) products
- Data is encrypted with [SSL/TLS](#)
- You can customize how HTTP traffic is handled with [rules](#) and [Workers Routes](#)
- Not sure where to start? Follow our [learning path](#) 

Your traffic data will show under [Analytics & Logs](#) once available.

]Screenshot: Cloudflare zone status **Active** (1 pt)

[

Q Search DNS Records Filters Display options Import Export + Add record									
You have used 8 of 200 available DNS records in this domain.									
①	Name ① ↕	Type ① ^	Content ① ↕	Proxy status ① ↕	TTL ① ↕	Tags	Comment	Details	
①	sargsyanush.site	CNAME	cf-pages-lab-12l.pages.dev	Proxied	Auto	—	—	—	Edit
	www.sargsyanu...	CNAME	cf-pages-lab-12l.pages.dev	Proxied	Auto	—	—	—	Edit
	sargsyanush.site	MX	eforward5.registrar-servers....	DNS only	Auto	—	—	20	Edit
	sargsyanush.site	MX	eforward4.registrar-servers....	DNS only	Auto	—	—	15	Edit
	sargsyanush.site	MX	eforward3.registrar-servers....	DNS only	Auto	—	—	10	Edit
	sargsyanush.site	MX	eforward2.registrar-servers....	DNS only	Auto	—	—	10	Edit
	sargsyanush.site	MX	eforward1.registrar-servers....	DNS only	Auto	—	—	10	Edit
	sargsyanush.site	TXT	"v=spf1 include:spf.efwd.reg...	DNS only	Auto	—	—	—	Edit
Showing 1 - 8 of 8									

] Screenshot: Cloudflare **DNS records** table for your zone (2 pts)

<https://sargsyanush.site/>

<https://www.sargsyanush.site/>] Working <https://yourdomain.xyz> and

<https://www.yourdomain.xyz> (3 pts)

Part 3 — DNS record questions (4 pts)

7.3 Fill in the table — what is each record type *for*? (2 pts)

Type	Purpose	Example value
A	Maps a domain or subdomain directly to an IPv4 address.	192.0.2.1
AAAA	Maps a domain or subdomain directly to an IPv6 address.	2001:0db8:85a3:0000:0000:8a2e:0370:7334
CNAME	Creates an alias, pointing one domain or subdomain to another domain name rather than an IP address.	cf-pages-lab-12l.pages.dev
MX	Specifies the mail servers responsible for accepting incoming email on behalf of the domain.	mail.example.com
TXT	Stores arbitrary text data for external services, used for domain verification and email security (SPF, DKIM, DMARC).	"v=spf1 include:spf.eforward.com ~all"

NS	Identifies the authoritative name servers responsible for managing the DNS records of the domain.	gerald.ns.cloudflare.com
-----------	---	--------------------------

7.4 Which record type(s) did Cloudflare create for your apex domain (`yourdomain.xyz`) and which for `www` ? Why can't a plain apex domain normally hold a CNAME — and what does Cloudflare do about it? (1 pt)

Cloudflare created **CNAME** records for both the apex domain (`yourdomain.xyz`) and the `www` subdomain.

DNS rules require the apex domain to have **SOA**, **NS**, and **MX** (email) records. A CNAME record overrides and deletes all other DNS records on the same hostname, which would **break email and core DNS**.

Cloudflare uses **CNAME Flattening**. It internally resolves the CNAME to its destination IP and serves an A/AAAA record to the public, keeping the domain fully standards-compliant.

7.5 Run these and paste the output: (1 pt)

`dig yourdomain.xyz +short`

```
104.21.49.250
172.67.197.216
```

`dig www.yourdomain.xyz +short`

```
172.67.197.216
104.21.49.250
```

`dig NS yourdomain.xyz +short`

```
heather.ns.cloudflare.com.
gerald.ns.cloudflare.com.
```

`curl -sI https://yourdomain.xyz | head -20`

```
HTTP/2 200
date: Mon, 07 Sep 2026 16:12:38 GMT
content-type: text/html; charset=utf-8
report-to:
{"group":"cf-nel","max_age":604800,"endpoints":[{"url":"https://a.nel.cloudflare.com/report/v4?s=Z4V%
2BP7VmeFCT4285vXIQZ%2FOPTIt0jWh%2BfDeWgEXNbgVWTWnFN83Lxo0o4p%2BVhM9OjLqTJ
d6ntty6LsYnG%2F7a0tNMNsIFNneQoEGNDWVKo7lr4xPWwgTunoPcOXBKeu2z53Ru"}]}
nel: {"report_to":"cf-nel","success_fraction":0.0,"max_age":604800}
access-control-allow-origin: *
cache-control: public, max-age=0, must-revalidate
x-content-type-options: nosniff
referrer-policy: strict-origin-when-cross-origin
server: cloudflare
cf-cache-status: DYNAMIC
cf-ray: a376ff05acd22af4-EVN
alt-svc: h3=":443"; ma=86400
```

Which header in the `curl` output proves the request went through Cloudflare?

server: cloudflare

Section 8 — VPS vs VDS (6 pts)

8.1 Define **VPS** and **VDS**. Then explain the real technical difference: which one **oversells/shares** CPU and RAM, and which **dedicates** it? (2 pts)

VPS (Virtual Private Server)	VDS (Virtual Dedicated Server)
A virtual server created using OS-level or lightweight virtualization where multiple isolated virtual instances share the underlying host machine's physical kernel and hardware resources. VPS oversells/shares CPU and RAM.	A virtual server created using hardware-level virtualization that runs its own completely isolated guest kernel and operates like a standalone physical machine. VDS dedicates CPU and RAM.

8.2 Complete the table: (2 pts)

	Shared hosting	VPS	VDS	Dedicated server
Root access	no	yes	yes	yes
Resources guaranteed	no	no	yes	yes
Typical monthly cost	low(2\$-10\$)	medium(\$5-\$20)	high(\$20-\$80)	very high(\$80+)
You patch the OS	no	yes	yes	yes

8.3 Pick VPS or VDS for each and justify in one line: (1 pt)

a) A hobby Telegram bot

VPS - It has low resource requirements and non-critical performance, making an inexpensive, oversold VPS the most cost-effective choice.

b) A production PostgreSQL database with strict latency requirements

VDS - Dedicated, non-oversold CPU and RAM guarantee predictable disk I/O and low-latency query performance without interference from "noisy neighbors."

8.4 You buy a "4 vCPU" VPS and it feels slower than advertised. Name **two** likely causes and **one** command

you'd run to investigate. (1 pt)

Two Likely Causes

- **CPU Overselling & Neighbor Noise:** The hosting provider oversold the physical CPU cores among too many virtual instances, causing vCPUs to wait for physical processor time.
- **Low Single-Core Frequency / Legacy Hardware:** The host server is using older, low-clock-speed CPU hardware, or the provider is severely throttling/limiting the maximum clock speed per vCPU core.

Command to Investigate - top, high %st value directly proves that the hypervisor is taking CPU cycles away from VM to serve other oversold tenants.

Section 9 — Git (8 pts)

9.1 Write the exact command for each: (3 pts)

Task	Command
a	Create a new branch and switch to it - <code>git switch -c <branch-name></code>
b	Stage only part of a file - <code>git add -p <file-name></code>
c	See the last 5 commits, one line each - <code>git log -n 5 --oneline</code>
d	Undo the last commit but keep the changes staged - <code>git reset --soft HEAD~1</code>
e	Discard all uncommitted changes to a tracked file - <code>git restore <file-name></code>
f	See what changed between your branch and main - <code>git diff main...HEAD</code>

9.2 Explain the difference between `git fetch` and `git pull` in one sentence. (1 pt)

git fetch downloads new commits, branches, and tags from the remote repository into your local copy without altering your working directory
git pull automatically runs `git fetch` and then immediately merges those downloaded changes into your current active branch

9.3 `merge` vs `rebase` : one advantage of each, and one rule about when **not** to rebase. (2 pts)

Advantage of Merge: It preserves the complete historical reality of the project by keeping all original commit timestamps, commit IDs, and feature branch structures fully intact.

Advantage of Rebase: It produces a clean, linear, and readable project history without cluttering the log with unnecessary merge commits.

When NOT to Rebase: Never rebase a shared or public branch that has already been pushed to a remote repository and is actively being used by other team members, as rewriting shared history will break local workflows and create severe sync conflicts.

9.4 You committed a `.env` file with a real password and pushed it. List the steps you take, **in order**. (1 pt)

1.Rotate/Invalidate the Credentials Immediately	Change the exposed password/secret in the system or service
2. Purge the File from Git History	git-filter-repo to completely erase the .env file and its history across all commits
3. Add .env to .gitignore	Git skips tracking it in the future, then make a new commit
4.Force Push the Cleaned History	Force push the rewritten, clean history to all remote branches
5.Notify the Team	Inform team members to delete their local copies of the repository and re-clone it fresh to prevent re-uploading the purged commit history.

9.5 Write a good and a bad commit message for the same change, and say what makes the good one good. (1 pt)

Bad Commit Message - bugfix, updates, changes

Good Commit Message - fix: resolve crash when querying missing DB table

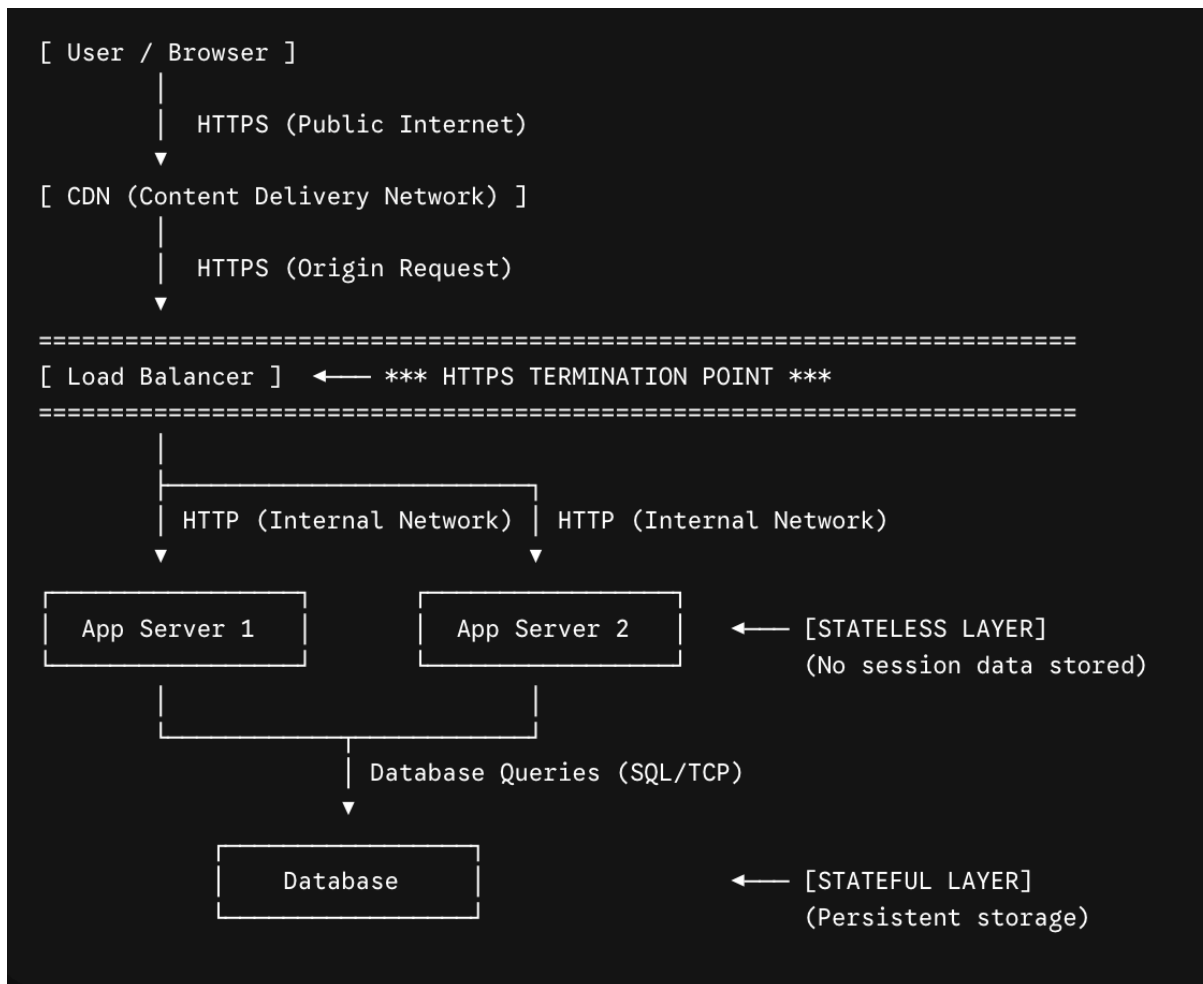
It states **what** was changed and **why**.

Section 10 — Three-Tier Application (8 pts)

10.1 Name the three tiers and give concrete technology for each. (2 pts)

Presentation Tier (Frontend)	React (or Next.js)
Application Tier (Backend / Logic)	Node.js with Express (or Python with FastAPI)
Data Tier (Database / Storage)	PostgreSQL (or MongoDB / MySQL)

10.2 Draw the architecture as ASCII art or a Mermaid diagram. Include: user → CDN → load balancer → app servers (×2) → database. Mark **where HTTPS terminates** and **which components are stateless**. (3 pts)



10.3 Why is separating the tiers useful? Give one **scaling** reason and one **security** reason. (1 pt)

Separating tiers is useful because each layer has a single responsibility. We can easily add more App Servers during traffic spikes without touching or over-provisioning the Database. The Database is hidden in a private network, blocking direct internet access.

10.4 Traffic doubles. Which tier do you scale first, and why is the database usually the hardest? (1 pt)

We must scale the **App Server** tier first, because app servers are stateless, we can immediately add more instances behind the Load Balancer (horizontal scaling) to distribute the CPU/RAM load from incoming requests without downtime. Unlike stateless servers, we can't just plug in extra database nodes - doing so requires complex setups like replication or sharding, which introduce major issues with **data sync, write conflicts, and latency**.

10.5 Map your Cloudflare Pages site from Lab A onto this model. Which tiers exist, and which are missing? (1 pt)

Existing Tiers:

CDN / Edge Tier: Cloudflare operates as a global CDN, natively handling HTTPS termination, DNS, and static asset caching.

Static App Tier: Your site's static files (HTML/CSS/JS) are served directly from Cloudflare's edge servers.

Missing Tiers:

Dedicated Load Balancer: Not required, as Cloudflare's edge network handles traffic routing automatically.

Dedicated App Servers: There are no traditional, persistent backend servers (e.g., Node.js or Python VMs) executing application logic.

Database Tier: There is no database tier attached, as the site only delivers static content.

Section 11 — Interview Preparation (8 pts)

11.1 — Scenario, written answer. Answer as if speaking to an interviewer, ~150–200 words: (3 pts)

"A user reports our website is down. You open it and it loads fine for you. Walk me through how you investigate."

Your answer must state your assumptions out loud and mention at least one tool.

1. If the site loads for me, my local browser might be serving a cached version or using a different DNS route.

2. I'll verify global availability using external tools (like VPN) to see if the issue is widespread or isolated to a specific region or ISP.

3. I'll check our observability dashboards and logs (like CloudWatch or Datadog) to determine whether it's an application/database bottleneck or a network/DNS failure.

If global probes confirm regional downtime, it's a network layer issue; if I see 5xx errors in the logs, I'll shift focus directly to origin server health."

11.2 — Scenario. (2 pts) *"You deployed 20 minutes ago. Error rate went from 0.1% to 4%.*

What do you do?"

Answer in 5 numbered steps. Say explicitly at which step you'd roll back.

Prioritize Mitigation Over Debugging: The immediate goal is to restore service stability for users, not to debug the new code in production.

Execute an Immediate Rollback (STEP 2 - ROLLBACK): Since the error rate spiked sharply from 0.1% to 4% right after the deployment, I will immediately initiate a rollback to the last known stable version.

Verify System Recovery: After the rollback completes, I'll check our monitoring metrics to confirm the error rate drops back down to the baseline 0.1%.

Conduct Root Cause Analysis (RCA): Once production is stable, I'll inspect the rolled-back deployment logs, stack traces, and APM metrics to pinpoint the exact bug or configuration flaw.

Fix, Test, and Redeploy: I'll replicate and patch the issue in a staging environment, run automated tests, and prepare a safe redeployment.

11.3 — Your questions for them. Write **four** questions you would ask the interviewer: one about the team, one about the on-call rotation, one about the tech stack, one about how success is measured in the first 6 months. (2 pts)

About the Team:	Will I have a mentor to help me learn the setup during onboarding?
About On-Call:	How does the team support new junior engineers before they join on-call?
About Tech Stack:	Which core tool-like Docker, Terraform, or CI/CD-will I work on most at first?
About Success:	What is the single most important milestone for me to reach in my first 3 months?

11.4 Name one thing you should **never** do in a technical interview when you don't know the answer — and what to do instead. (1 pt)

Never lie, fake an answer, or pretend I know something I don't.
Admit honestly that I don't know, explain my logical problem-solving approach, and mention how I would quickly learn it using documentation or past experience with similar concepts.